Comparative Acceleration of Optical Flow Estimation on Heterogeneous Embedded Platforms: FPGA vs. Embedded GPU

George Briceño Celestino
Escuela de Ingeniería Electrónica
Instituto Tecnológico de Costa Rica
Cartago, Costa Rica
rousell@estudiantec.cr

Laura Elena Brenes Espinoza
Escuela de Ingeniería Electrónica
Instituto Tecnológico de Costa Rica
Alajuela, Costa Rica
lauritabrenes@estudiantec.cr

Marlon Méndez Naranjo
Escuela de Ingeniería Electrónica
Instituto Tecnológico de Costa Rica
San Carlos, Costa Rica
mnaranjo@estudiantec.cr

Rodrigo Venegas Mora
Escuela de Ingeniería Electrónica
Instituto Tecnológico de Costa Rica
Alajuela, Costa Rica
rovenegas@estudiantec.cr

***Index Terms*—Optical flow, Heterogeneous computing, FPGA, Embedded GPU, AMD Kria, NVIDIA Jetson**

I. Introduction

Optical flow estimation is one of the fundamental primitives of computer vision, enabling the computation of apparent pixel-level motion between consecutive image frames. This capability is critical for applications such as autonomous navigation, robotic obstacle avoidance, video stabilization, and augmented reality systems [4]. As these applications increasingly migrate toward embedded and edge platforms with strict power and latency constraints, the need for efficient hardware acceleration of optical flow algorithms has become a pressing research challenge.

Classical optical flow methods, including Lucas-Kanade (LK) [39], Horn-Schunck (HS) [1], and Total Variation L1 (TV-L1) [16], are grounded in the Brightness Constancy Constraint and employ variational or local parametric formulations to estimate dense or sparse motion fields. These methods exhibit highly regular computational structures — gradient computations, iterative solvers, and multi-scale pyramids — that map efficiently onto parallel hardware architectures. In contrast, deep learning-based approaches such as RAFT [7], FlowFormer++ [10], and NeuFlow_v2 [3] have dramatically surpassed classical methods in accuracy, with the current state of the art reaching an End-Point Error (EPE) of 0.963 pixels on the Sintel Clean benchmark [10]. However, deploying these models on resource-constrained embedded hardware remains an open challenge due to their reliance on complex tensor operations, large memory footprints, and iterative refinement stages. The current state of the art, FlowFormer++, achieves 1.07 pixels End-Point Error (EPE) on the Sintel Clean benchmark and 1.94 EPE on Sintel Final [10].

Heterogeneous computing platforms that combine ARM processors, FPGAs, and embedded GPUs offer a promising path to bridge the gap between algorithmic accuracy and real-time embedded performance. FPGA implementations using High-Level Synthesis (HLS) have demonstrated exceptional energy efficiency for classical optical flow algorithms, with Horn-Schunck and Lucas-Kanade designs achieving 1080p60 on Xilinx Zynq UltraScale+ devices under 6 W [1], [12]. On the other hand, NVIDIA Jetson platforms provide superior flexibility for deep learning deployment through CUDA and TensorRT, with NeuFlow_v2 achieving over 20 FPS on Jetson Orin Nano [3].

Despite the maturity of individual platform implementations, the literature reveals a significant gap no direct comparative benchmark exists between the AMD Kria KV260 FPGA platform and the NVIDIA Jetson Nano for optical flow estimation. Existing studies report results on generic Zynq-7000 or Zynq UltraScale+ platforms without targeting the specific KV260 reference design, while Jetson benchmarks focus predominantly on the TX2, Xavier, and Orin generations with sparse coverage of the Nano [12], [24]. Furthermore, the AMD Vitis Vision Library lacks validated reference designs for optical flow on the Kria KV260, representing an additional contribution opportunity [6], [16].

This work proposes a rigorous comparative evaluation of classical optical flow algorithms — specifically Dense Pyramidal Lucas-Kanade and TV-L1 — on the AMD Kria KV260 (Zynq UltraScale+ MPSoC) and NVIDIA Jetson Nano (Tegra X1 Maxwell) platforms. The evaluation encompasses multiple resolutions (VGA, 720p, 1080p), standard validation benchmarks (Middlebury, KITTI 2015), and systematic measurement of performance metrics including frames per second, per-frame latency, average power consumption, energy per frame, and computational efficiency. The contribution aims to

provide the first published direct comparison between these two widely available embedded platforms for optical flow, offering practitioners and researchers an objective basis for platform selection in real-time vision applications.

The remainder of this paper is organized as follows: Section II reviews the background and related work on optical flow algorithms and heterogeneous platforms.

II. Background and Related Work

A. Optical Flow Fundamentals

Optical flow estimation is grounded in the Brightness Constancy Constraint (BCC), which assumes that pixel intensity remains constant between consecutive frames, yielding the fundamental equation

$$I_x u + I_y v + I_t = 0, \tag{1}$$

where I_x and I_y denote spatial intensity gradients, I_t is the temporal gradient, and (u, v) represents the flow vector components [4]. Since this single equation is underdetermined for two unknowns per pixel, classical methods are distinguished by their regularization strategy. Local methods, exemplified by Lucas-Kanade (LK) [39], assume uniform motion within a spatial neighborhood and solve an overdetermined least-squares system per pixel, offering inherent per-pixel parallelism ideal for hardware acceleration but limited to textured regions and small displacements — a limitation addressed by multi-scale pyramidal extensions. Global methods, such as Horn-Schunck (HS) [1], impose a smoothness constraint across the entire image through variational energy minimization, producing dense flow fields via iterative solvers (Jacobi/Gauss-Seidel) whose regular structure maps efficiently onto FPGA line-buffer architectures. Variational methods like TV-L1 [16] replace the quadratic data term with an L1 norm, achieving greater robustness to outliers and occlusions through dualization (Chambolle-Pock), and are available as optimized HLS kernels in the AMD Vitis Vision Library [6], [16]. More recently, deep learning approaches — from FlowNet2 [2] and PWC-Net [8] to RAFT [7] and FlowFormer++ [10] — have dramatically surpassed classical methods in accuracy by learning hierarchical feature representations, with FlowFormer++ achieving 1.07 EPE on Sintel Clean and 1.94 EPE on Sintel Final [10]. However, their deployment on embedded platforms with less than 15 W TDP remains an active research challenge due to memory bandwidth constraints and iterative refinement overhead [3], [11].

B. Classical Methods on Heterogeneous Platforms

Implementations in FPGA: Classical optical–flow algorithms on FPGA consistently achieve superior energy efficiency and deterministic latency. Komorkiewicz [1] demonstrates high–throughput Horn–Schunck on Zynq–7000 with favorable energy per frame, while Blachut and Kryjak [12] report multi–scale Lucas–Kanade and Horn–Schunck designs scaling up to validated 4K@60 FPS under 6 W [12]. Production–ready HLS kernels for Dense Pyramidal LK and TV–L1 are available in the Vitis Vision Library [6], [16], though no complete, publicly validated optical–flow reference design exists for the Kria KV260. At the opposite end of the design space, BNN–based approaches such as Ultra–Flow reach hundreds of FPS at VGA resolution with reduced accuracy [15]. These results motivate benchmarking classical kernels (Dense Pyramidal LK, TV–L1) on KV260 and embedded GPUs to quantify throughput, energy per frame, and estimation quality.

Implementations in embedded GPU: Embedded GPUs provide flexible deployment of classical and learned optical–flow methods, with performance strongly tied to device generation and available accelerators. On Jetson Nano, classical methods (Farnebäck, TV–L1) run via OpenCV+CUDA or VPI in CPU/CUDA mode because the Nano lacks NVIDIA's Optical Flow Accelerator (OFA) [19], [23]. Higher–end Jetson devices (AGX Xavier, Orin) support dense optical flow at 60 FPS and efficient inference of lightweight DNNs using TensorRT (e.g., FastFlowNet, NeuFlow_v2), achieving tens of FPS through kernel fusion and quantization [24], [26], [27]. In practice, GPUs accelerate development and excel for DNN–based flow, while FPGAs offer better energy per frame and deterministic latency for classical kernels. A controlled comparison measuring FPS, per–frame latency, average power, energy/frame, and EPE (Middlebury, KITTI) is therefore required for informed platform selection.

C. Deep Learning Approaches for Optical Flow

The evolution of deep learning-based optical flow follows: FlowNet2 [2] → PWC-Net [8] → RAFT [7] → FlowFormer++ [10]. RAFT achieves 5.10% Fl-all on KITTI-2015 through recurrent all-pairs field transforms, while FlowFormer++ reaches 1.94 EPE on Sintel Final via masked cost volume autoencoding [10]. Lightweight models targeting edge deployment include SEA-RAFT [11] ($\sim$1.3M parameters) and NeuFlow_v2 [3] ($\sim$2.5M parameters, $>$20 FPS on Jetson Orin Nano). However, DNN deployment on platforms below 15 W TDP remains challenging due to memory bandwidth constraints and iterative refinement overhead [3], [11].

D. ARM NEON Optimizations

ARM NEON is a SIMD (Single Instruction, Multiple Data) extension which is included in many ARM Cortex-A processors, this because instead of executing an operation on a single piece of data, it allows the same operation to be applied to several data simultaneously using vector registers of up to 128 bits [38]. For example, in a multiplication operation for an algorithm in scalar processing one operation would be done at a time, but in the case of NEON it can be done by one in a vector operation, multiple elements can be processed simultaneously using vector instructions [30]. This is especially useful in image processing, as the exact same operations are repeated on thousands or millions of pixels [30], [38]. For ARM, it is precisely called image processing, video, audio, and signal processing as suitable loads for NEON [38].

Now, in the case of the Lucas–Kanade (LK) algorithm, it looks for an optical Flow, which tries to determine how it moved from a point between two consecutive images.

The Lucas–Kanade (LK) method estimates optical flow by determining the apparent displacement of image points between consecutive frames [39]. For this algorithm there are several stages that are excellent candidates for SIMD, such as: Lucas–Kanade standalone, LK optical-flow pyramid builder, pyramidal LK, Scharr, multi-channel blur, downsampling, derivatives, iterative tracking [5].

NEON improves performance by using SIMD instructions, which allow the processor to apply the same operation to multiple data elements, such as several pixel values, in parallel instead of processing them one by one. KleidiCV 26.03 takes advantage of this type of Arm architecture feature through optimized computer vision kernels, and its optimizations are not limited to the Lucas–Kanade (LK) core, but can also cover other stages of the sparse optical flow pipeline, including pyramid construction, Scharr filtering, blurring, downsampling, and pyramidal LK [5]. Therefore, KleidiCV and NEON should not be considered the same technology: NEON is an Arm SIMD instruction-set extension available on supported processors, while KleidiCV is a software library that provides optimized implementations of computer vision operations and can select or use implementations designed for different Arm architecture extensions, such as NEON, SVE2, SME, and SME2, depending on the target hardware. [5].

This merely depends on the available hardware. In particular, the normal APIs use NEON or SVE2 by default, while the "sme" variants can use SME or SME2 [5]. In addition, it should be noted that these are already integrated with OpenCV, which ARM indicates that since OpenCV 4.13, KleidiCV are enabled by default for AArch64 builds on Android, Linux and macOS [5], in order to reduce the execution time of image processing operations.

It is important to keep in mind that the different optimizations using NEON should be considered as a complement to higher capacity accelerators, such as GPUs and FPGAs, since their main advantage is that the SIMD instructions are executed directly on the ARM processor, in order to avoid additional data transfers and allowing to accelerate preprocessing, post-processing stages or algorithm components that do not justify their execution in a dedicated accelerator [25], [30]. For this reason, NEON is very important in the different heterogeneous architectures and embedded systems where computational efficiency, energy consumption and balanced use of resources are important factors.

E. Research Gap and Positioning

Existing optical flow benchmarks on embedded platforms focus predominantly on high-end devices: NeuFlow_v2 reports 20+ FPS on Jetson Orin Nano (15 W TDP) but provides no comparison to FPGA implementations [3], while Seznec et al. [24] achieve 60 FPS dense optical flow on Jetson AGX Xavier (30 W) through operator fusion and pipeline optimization, yet their TV-L1 implementation strategy cannot be directly transferred to the Jetson Nano's Maxwell architecture lacking the Optical Flow Accelerator present in Turing and Ampere GPUs [19], [23]. On the FPGA side, Blachut and Kryjak [12], [35] validate multi-scale Lucas-Kanade and Horn-Schunck designs at 4K@60 FPS on Zynq UltraScale+ devices, but their work targets generic evaluation boards rather than the Kria KV260's specific camera interface, memory topology, and thermal envelope.

Furthermore, while the AMD Vitis Vision Library provides production-ready HLS kernels for Dense Pyramidal Lucas-Kanade and TV-L1 [6], [16], no complete, publicly validated optical flow reference design exists for the Kria KV260 Vision AI Starter Kit. Similarly, Jetson Nano benchmarks remain sparse compared to higher-end devices (TX2, Xavier, Orin), with most studies focusing on deep learning approaches rather than classical methods [24], [37].

This work addresses these gaps by providing a comparative benchmark between the Kria KV260 (Zynq UltraScale+ MPSoC) and Jetson Nano (Tegra X1 Maxwell) for classical optical flow estimation, specifically Dense Pyramidal Lucas-Kanade and TV-L1 algorithms across multiple resolutions (VGA, 720p, 1080p) and standard validation benchmarks (Middlebury, KITTI 2015).

III. Project Progress 1

A. Study of Other Solutions

Strengths and weaknesses of FPGA: existing designs, such as those by Komorkiewicz [1] and Blachut and Kryjak [12], demonstrate that Lucas-Kanade and Horn-Schunck on FPGA achieve high energy efficiency and deterministic latency, reaching up to 4K at 60 frames per second with less than 6 watts. AMD's Vitis Vision library [6] already offers optimized HLS kernels for pyramidal Lucas-Kanade and TV-L1. However, the weakness is clear: these results are validated on generic Zynq boards, not on the Kria KV260, and no complete published reference design exists for that specific board.

Strengths and weaknesses of embedded GPU: NVIDIA's ecosystem [19, 23] offers development flexibility through CUDA and VPI, and on higher-end platforms, as demonstrated by Seznec [24], 60 frames per second of dense optical flow are achieved. The specific weakness is that these results correspond to Xavier and Orin, while the Jetson Nano, due to its Maxwell architecture, lacks a dedicated optical flow accelerator, forcing brute-force execution on generic CUDA.

Strengths and weaknesses of deep learning: models such as RAFT [7] and FlowFormer++ [10] achieve outstanding accuracy, with FlowFormer++ reaching 0.963 pixels of error on Sintel Clean. Even lightweight versions such as SEA-RAFT [11] and NeuFlow v2 [3] attempt to reduce model size. The weakness is that, even when reduced, they still demand more memory and computation than the Jetson Nano or the Kria KV260 can efficiently provide below 15 watts.

Feasibility: from this analysis it follows that classical methods, Lucas-Kanade and TV-L1, are the only ones viable for efficient deployment on both platforms within the project's power budget, while deep learning is not feasible under those constraints.

Observed challenges:

- First, the absence of a validated optical flow reference design for the Kria KV260.
- Second, the absence of comparable data between FPGA and embedded GPU under the same metrics and the same algorithm.
- Third, scarce coverage of the Jetson Nano in the literature, mostly concentrated on higher Jetson generations.

B. Proposed Solution

A comparative test bench is proposed that implements Dense Pyramidal Lucas-Kanade and TV-L1 on the two heterogeneous computing platforms under study, the AMD Kria KV260 and the NVIDIA Jetson Nano, following an identical three-stage methodology on both: a baseline in C with NEON optimizations on the ARM processor of each board, followed by a platform-specific acceleration stage—HLS on the FPGA in the case of the Kria and CUDA on the GPU in the case of the Jetson.

How it addresses the first challenge, the absence of a validated reference design for the KV260: a custom HLS implementation of the pyramidal Lucas-Kanade and TV-L1 kernels is built, starting from the blocks available in the Vitis Vision library [6], adapted and validated specifically on the camera interface, memory topology, and thermal envelope of the Kria KV260, thereby closing the gap identified by Blachut and Kryjak [12] when working on generic boards.

How it addresses the second challenge, the absence of comparable data under the same conditions: by applying the same three-stage methodology and the same set of metrics—frames per second, latency per frame, average power, energy per frame, and estimation quality using the Middlebury and KITTI 2015 benchmarks—on both platforms, a directly comparable dataset between FPGA and embedded GPU is generated for the first time for these algorithms, something the current literature does not offer because each prior study [1, 12, 24] measures under different conditions and boards.

How it addresses the third challenge, the scarce coverage of the Jetson Nano: the behavior of Lucas-Kanade and TV-L1 on the Nano's Maxwell architecture is explicitly documented, lacking the dedicated optical flow accelerator (OFA) present in higher generations [19, 23], thereby generating the first reference point focused on this specific platform instead of extrapolating from Xavier or Orin [24].

Scope: the project is limited to the two classical algorithms mentioned, at three resolutions—VGA, 720p, and 1080p—evaluated on the Middlebury and KITTI 2015 validation sets. The implementation of deep learning methods falls outside the scope, given that, as established in the study of other solutions, they are not feasible under the power constraints of both platforms.

C. Proposed Architecture

Figure 2 shows the Level II architecture of the Kria KV260. The system is divided into two physically separated domains: the Processing System (PS), based on a quad-core ARM Cortex-A53 processor where the NEON baseline runs, and

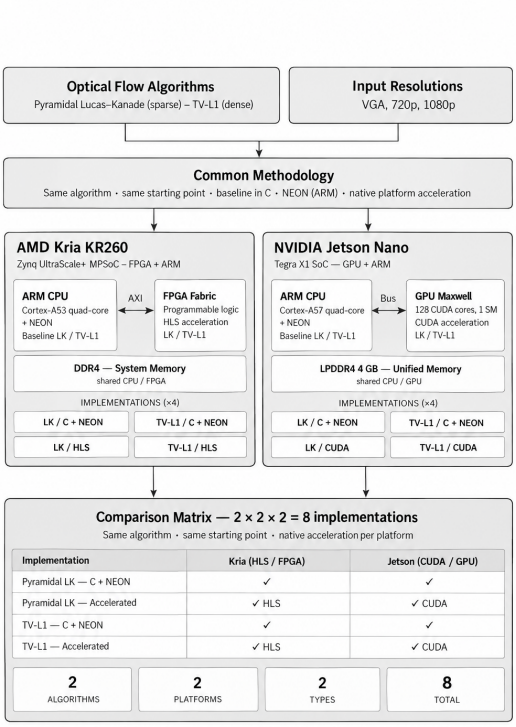


Fig. 1. Comparative architecture and implementation matrix for optical flow evaluation on AMD Kria KR260 and NVIDIA Jetson Nano.

the Programmable Logic (PL), where the HLS kernels for pyramidal Lucas-Kanade and TV-L1 derived from the Vitis Vision library are implemented. Both domains communicate via AXI interconnect and share 4 GB of DDR4 memory. Input frames (dataset or camera) enter the system, are processed in PS or PL depending on the stage being evaluated, and the resulting flow field—a displacement vector (u, v) per pixel—is extracted along with the timestamps needed for latency calculation.

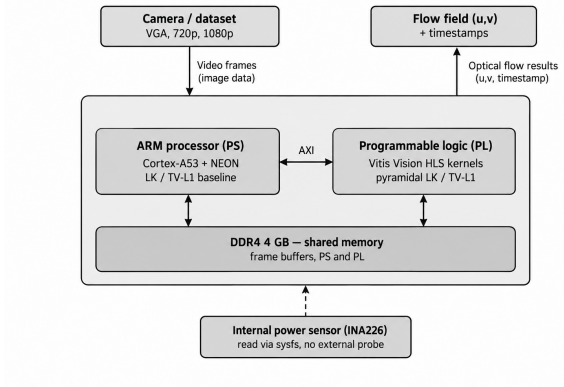


Fig. 2. Level II hardware architecture and data flow for optical flow processing on the Kria KV260.

Figure 3 shows the Level II architecture of the Jetson Nano.

Unlike the Kria, the Tegra X1 integrates a quad-core ARM Cortex-A57 CPU and a Maxwell GPU with a single streaming multiprocessor (128 CUDA cores) on a unified LPDDR4 memory architecture of 4 GB, with a peak bandwidth of 25.6 GB/s. The NEON baseline runs on the CPU and native acceleration is implemented as CUDA kernels on the GPU. Unlike higher Jetson generations, the Nano does not have a dedicated optical flow accelerator (OFA), so acceleration relies exclusively on general-purpose CUDA computation.

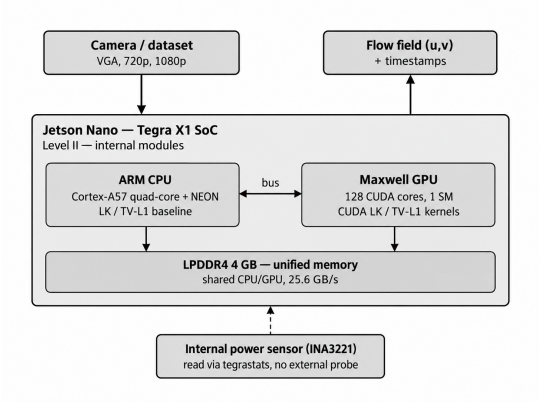


Fig. 3. Level II hardware architecture and data flow for optical flow processing on the Jetson Nano.

D. List of Experiments

Four experimental blocks are proposed, executed under controlled temperature and power supply conditions on both platforms, so that measured differences are attributable to the architecture and not to experimental variability. First, accuracy evaluation: pyramidal Lucas-Kanade and TV-L1 are run on the Middlebury and KITTI 2015 validation sets, comparing the estimated flow against the reference flow, to isolate the algorithm's own error from the error introduced by each hardware implementation. Second, resolution sweep: each algorithm is repeated at VGA, 720p, and 1080p, to observe how performance and power consumption scale with image size, and to distinguish whether the bottleneck is computation or memory bandwidth on each platform. Third, comparison between acceleration stages: each algorithm is measured in its NEON baseline version against its native accelerated version—HLS on the Kria and CUDA on the Jetson—to quantify the speedup factor and the power cost introduced by each dedicated accelerator. Fourth, cross-platform comparison: Kria is directly contrasted against Jetson under the same algorithm and resolution conditions, which is the central result of the project.

- **Throughput:** frames processed per second at steady state, measured after the thermal preconditioning period, excluding initial data loading time and accelerator initialization time.
- **Processing time per frame (ms):** measured end-to-end, from when the input frame is available in memory until the output flow is ready; mean and 95th percentile are reported to capture jitter.
- **Average power:** obtained via software from the integrated rail monitoring sensors on each board (INA226 on the Kria KV260, INA3221 on the Jetson Nano), queried via console (sysfs / tegrastats) during the sustained execution window, without additional physical instrumentation.
- **Energy per frame:** calculated as average power divided by throughput, in joules per frame, to normalize energy efficiency between platforms with different nominal power and different operating regimes.
- **Estimation quality (EPE, end-point error):** the average Euclidean error between the estimated flow vector and the reference flow vector, calculated only over valid pixels (ground truth mask) of each dataset, expressed in pixels.

REFERENCES

[1] M. Komorkiewicz, M. Kluczewski, and P. Skruch, "Floating-Point versus Fixed-Point Optical Flow on FPGA," *Sensors*, vol. 14, no. 2, pp. 2860–2891, 2014. https://www.mdpi.com/1424-8220/14/2/2860

[2] A. Dosovitskiy, P. Fischer, E. Ilg, et al., "FlowNet: Learning Optical Flow with Convolutional Networks," arXiv preprint arXiv:1504.06852, 2015. https://arxiv.org/abs/1504.06852

[3] L. Kong and Z. Shen, "NeuFlow_v2: High-Efficiency Optical Flow Estimation on Edge Devices," arXiv:2408.10161, 2024. https://arxiv.org/abs/2408.10161

[4] Wikipedia contributors, "Optical flow," *Wikipedia, The Free Encyclopedia*, 2024. https://en.wikipedia.org/wiki/Optical_flow

[5] ARM Ltd., "What's new in KleidiCV 26.03 for Computer Vision on Arm CPUs," *ARM Developer Blog*, March 2026. https://developer.arm.com/community/arm-community-blogs/b/ai-blog/posts/what-s-new-in-kleidicv-26-03-for-computer-vision-on-arm-cpus

[6] Xilinx / AMD, "Vitis Vision Library," 2022. https://docs.amd.com/r/en-US/Vitis_Libraries/vision/index.html

[7] Z. Teed and J. Deng, "RAFT: Recurrent All-Pairs Field Transforms for Optical Flow," arXiv:2003.12039, 2020. https://arxiv.org/abs/2003.12039

[8] D. Sun, X. Yang, M.-Y. Liu, and J. Kautz, "PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume," arXiv:1709.02371, 2018. https://arxiv.org/abs/1709.02371

[9] Z. Huang et al., "FlowFormer: A Transformer Architecture for Optical Flow," arXiv:2203.16194, 2022. https://arxiv.org/abs/2203.16194

[10] H. Shi et al., "FlowFormer++: Masked Cost Volume Autoencoding for Pretraining Optical Flow Estimation," arXiv:2303.01237, 2023. https://arxiv.org/abs/2303.01237

[11] S. Wang et al., "SEA-RAFT: Simple, Efficient, Accurate RAFT for Optical Flow," arXiv:2405.14793, 2024. https://arxiv.org/abs/2405.14793

[12] P. Blachut and T. Kryjak, "Real-Time Efficient FPGA Implementation of the Multi-Scale Lucas-Kanade and Horn-Schunck Optical Flow Algorithms for a 4K Video Stream," *Sensors*, vol. 22, no. 13, p. 5017, 2022. https://www.mdpi.com/1424-8220/22/13/5017

[13] P. Blachut and T. Kryjak, "Multi-scale Lucas-Kanade Optical Flow on FPGA for Surveillance," in *Proc. DASIP*, 2018. https://www.researchgate.net/publication/325306206

[14] S. Chang et al., "FPGA Implementation of Optical Flow Using High-Level Synthesis," in *Proc. ASAP 2013*, NSF SHREC, 2013. https://ieeexplore.ieee.org/document/6601837

[15] A. Author et al., "Ultra-Flow: Real-Time Optical Flow on FPGA with Binary Neural Networks," *Springer Signal, Image and Video Processing*, 2025. https://link.springer.com/journal/11760

[16] AMD/Xilinx, "Vitis Vision Library Design Examples," 2022. https://github.com/Xilinx/Vitis_Libraries/tree/master/vision

[17] AMD, "Vitis Vision API Reference: densePyrOpticalFlow," 2022. https://docs.amd.com/r/en-US/Vitis_Libraries/vision/methods.html

[18] Xilinx, "Dense Pyramidal LK Optical Flow Benchmark," 2022. https://github.com/Xilinx/Vitis_Libraries/tree/master/vision/examples

[19] NVIDIA, "DeepStream SDK Plugin: gst-nvof," *NVIDIA Developer*, 2023. https://developer.nvidia.com/optical-flow-sdk

[20] NVIDIA Developer Blog, “OpenCV Optical Flow Algorithms with NVIDIA Turing GPUs,” 2019. https://developer.nvidia.com/blog/opencv-optical-flow-algorithms-with-nvidia-turing-gpus/

[21] NVIDIA, “Optical Flow SDK Download,” 2024. https://developer.nvidia.com/optical-flow-sdk

[22] NVIDIA Developer Blog, “Harnessing the NVIDIA Ada Architecture for Frame Rate Up-Conversion in the NVIDIA Optical Flow SDK,” 2023. https://developer.nvidia.com/blog/harnessing-nvidia-ada-architecture-for-frame-rate-up-conversion/

[23] NVIDIA, “VPI — Vision Programming Interface: Basic Concepts,” 2023. https://docs.nvidia.com/vpi/

[24] R. Seznec et al., “Real-Time Optical Flow Processing on Embedded GPU: a Hardware-Aware Algorithm to Implementation Strategy,” *Journal of Real-Time Image Processing*, 2022. https://hal.science/hal-03457011/document

[25] E. Carballo-Hernández et al., “Heterogeneous Acceleration of CNN Inference on Embedded FPGA-GPU,” *Journal of Real-Time Image Processing*, 2021. https://www.springer.com/journal/11554

[26] L. Kong et al., “FastFlowNet: A Lightweight Network for Fast Optical Flow Estimation,” GitHub, 2021. https://github.com/liyaguang/fastflownet

[27] NVIDIA Developer Blog, “Speed Up Inference with TensorRT,” 2022. https://developer.nvidia.com/blog/speed-up-inference-with-tensorrt/

[28] OpenCV GitHub Issue #6979 — “NEON optimizations for calcOpticalFlowPyrLK on ARM.” https://github.com/opencv/opencv/issues/6979

[29] Unknown Author, “ARM NEON Optical Flow Acceleration,” *CEUR Workshop Proceedings*, vol. 2588, Paper 37, 2020. https://ceur-ws.org/Vol-2588/paper37.pdf

[30] E. McCreath, “Use of SIMD Vector Operations to Accelerate Application Code Performance on Low-Power ARM and Intel Platforms,” ANU Technical Report, 2018. https://comp.anu.edu.au/courses/comp3220/lectures/2018/

[31] ARM Software (GitHub), “ARM Compute Library — OpticalFlowLK Issue #216.” https://github.com/ARM-software/ComputeLibrary/issues/216

[32] ResearchGate, “FLIA: Fast Lightweight Image Analysis via Heterogeneous Embedded Computing,” 2022. https://www.researchgate.net/publication/362044128

[33] A. Lacas et al., “Optical Flow FPGA Co-processor for Drones,” in *Proc. ASAP 2023*, 2023. https://hal.science/hal-04199828

[34] P. Blachut and T. Kryjak, “Real-time Optical Flow Estimation on FPGA for Traffic Surveillance,” in *Proc. DASIP*, 2018. https://www.researchgate.net/publication/325306206

[35] P. Blachut and T. Kryjak, “Real-time LK and HS Optical Flow on FPGA at 4K Resolution,” in *Proc. ECCV 2022 Workshop*, 2022. https://www.mdpi.com/1424-8220/22/13/5017

[36] S. Jiang, D. Campbell, Y. Lu, H. Li, and R. Hartley, “Learning to Estimate Hidden Motions with Global Motion Aggregation,” arXiv:2104.02409, 2021. https://arxiv.org/abs/2104.02409

[37] J. Jeong et al., “Embedded Systems Survey for Optical Flow,” *Electronics*, vol. 11, no. 22, p. 3756, 2022. https://www.mdpi.com/2079-9292/11/22/3756

[38] Arm Ltd., “Coding for Neon,” *Arm Neon Programmer’s Guide*, Issue 04, 2020. https://developer.arm.com/architectures/learn-the-architecture/neon

[39] B. D. Lucas and T. Kanade, “An Iterative Image Registration Technique with an Application to Stereo Vision,” *Proc. IJCAI*, 1981. https://www.cmu.edu/biolphys/dennis/notes/vision/lk.pdf